MDS Capstone Proposal - MDP Exception

Student: Yash Kulkarni
UM ID: 10435489
Program: MS in Data Science, University of Michigan
Supervisor: Prof. Abell
Course: ENGR 555 (WN26 + FA26, 2+2 credits)

Project Title

Interpretable Delay-Aware Deep Learning for Milling Chatter Prediction: A Temporal Kolmogorov-Arnold Network Verified Against Regenerative Delay Physics

Project Overview

Chatter is a self-excited vibration that limits how fast a milling machine can cut. Its governing physics is a delay differential equation: each tooth removes material left behind by the previous tooth, so the dynamics depend on the tool position now and its position one tooth-pass period ago. That delay, not the vibration amplitude, generates the instability.

A classifier trained on vibration signals can reach high accuracy here without anyone knowing whether it uses the delay structure or a spectral shortcut that correlates with it. This project trains a temporal Kolmogorov-Arnold network (TKAN) on the Schmitz digital machining database (155 simulated datasets, 33,719 windowed signal files) to classify chatter and predict its severity, then tests by intervention whether the network depends on the regenerative delay. The research question is whether a model given only vibration signals and process parameters recovers the physical delay, and whether that dependence can be shown causally rather than inferred from an attribution plot. That second half is what separates this work from existing machine-learning chatter classifiers, which report accuracy but leave the learned function untested against the physics.

Individual Contributions

I am responsible for the design, implementation and evaluation of the full pipeline:

Data Pipeline

- SINDy-DDE preprocessing and the Phase 2 spectral stage: power spectral density estimation and Jensen-Shannon divergence across the b/blim stability boundary, plus multi-family noise metrics and signal-mode assignment over the clean/noisy mirror ladder
- Building the Phase 3 windowed sequence dataset of 33,719 files with grouped five-fold cross-validation
- Replacing arithmetic parameter-group assignment with connected-component grouping over parameter identity. The original mirror-relation check missed two parameter-identical pairs (1 and 23, 16 and 17), which could place one operating point on both sides of a split
- Excluding 700 files with an invalid low-speed stability limit and 560 with unavailable metrics, with counts recorded per run so no file disappears silently

Model Architecture

- Writing the KANLinear layer from scratch (SiLU base plus B-spline, no pykan runtime dependency), passing inputs through tanh so the spline grid range matches z-scored data
- A KAN-GRU recurrent encoder over 1024-step sequences, four signal channels plus their delayed copies, feeding two staged heads: chatter classification and b/blim severity regression, trained jointly under weighted mean squared error and binary cross-entropy
- Process-parameter side inputs (spindle speed, tooth-pass frequency, delay) behind a guard that refuses any feature capable of leaking the label

Training at Scale on Great Lakes

- Profiling to locate the bottleneck. Per batch the model costs roughly 2,900 to 3,300 times an MLP, and at 1024 steps the recurrence is bound by GPU kernel-launch latency, not arithmetic. Compiling the recurrent cell gave a 3.4x speedup, 1,374 to 399 ms per batch, bringing one fold from about 30 hours to 8.5 hours on an A40
- Diagnosing a training instability that had been failing silently. At a peak learning rate of 0.003 the KAN-GRU gradients blew up (worst case 1,661 non-finite batches of 3,334 in one epoch), and the naive clipping call scaled every gradient by max_norm/NaN, turning the whole network to NaN before any metric could report it. Halving the rate to 0.0015 with a cosine schedule, warmup, clipping and AdamW gave zero non-finite batches across all 100 epoch-folds

Results

- Five-fold cross-validated AUC of 0.955 plus or minus 0.022, against a matched MLP baseline of 0.875 plus or minus 0.004 on the same folds, an improvement of 0.080 on every fold
- Evaluation-only AUC of 0.908 plus or minus 0.013 on five datasets present in no training fold, run with the trained checkpoints and no retraining
- On the strict held-out set (datasets 122-125, 855 files, no parameter twin anywhere in training), the model released to the team scores AUC 0.920 and accuracy 0.841. This is the figure I report to collaborators; the cross-validation number is inflated by the parameter-twin leakage above and I do not quote it as the headline

White-Box Verification

The contribution of the project, built on interventions rather than attribution maps.

- Delay ablation. Zeroing the delayed input channels drops chatter AUC from about 0.99 to 0.71 on fold 2, and to at or near chance on three of five folds (0.500, 0.510, 0.541); severity error rises from 0.17 to 11.6
- Per-window delay sweep. Accuracy peaks at exactly 1.0 times each window's own tooth-pass period on all five folds. Halving costs 0.12 to 0.16 AUC, doubling costs 0.04 to 0.06, so the network keys on the physical period and not a generic lag
- Delay jitter. Accuracy degrades monotonically as the stated delay is made wrong, 0.994 to 0.842 at 50 percent jitter on fold 2, giving a third independent causal line
- Symbolic extraction. The 25 highest-importance spline edges on every fold accept a closed-form expression under a BIC gate, fit R-squared 0.926 to 0.9999
- Model-implied stability lobe diagram. Sweeping the network's decision boundary over 1,014 speed-depth cells reproduces the analytic stability limit on 98.2 percent of cells, IoU 0.94, mean boundary error 0.35 mm. The network has recovered the classic lobe structure
- Independent physics recovery. A SINDy-DDE fit to the raw signals, which never sees the network, returns a delay 0.99 times physical with the regenerative term selected in every window: two independent methods on the same delay

- Network-grounded counterfactuals. A surrogate fit to the model's own predictions moves 38 of 42 chatter operating points to stable, 31 below the analytic limit, for 81.6 percent agreement with the physics
- Calibrated uncertainty. A conformal wrapper over the five-checkpoint ensemble reaches 88.3 percent coverage at a 90 percent target with 99.6 percent selective accuracy

Verification and Retracted Claims

A significant share of this project went into testing my own results; several did not survive.

- Repeatability floor. From ten duplicate run pairs I measured a floor of 0.0092 AUC, the largest gap between two runs of an identical configuration, and a lower bound since the pairs share a seed. The cause is the trainer seeding every RNG while never requesting deterministic algorithms under compilation, so kernel selection follows transient GPU state
- Two of my own findings fell under that floor: the side-input contribution at 0.0028 AUC and the quantile severity head at 0.0062. I withdrew an earlier claim that the quantile head was a real improvement, with a dated correction in the report that carried it
- A delay-lock statistic turned out to be a construction artifact. Temporal saliency rescaling appeared to lock onto the physical delay far beyond chance, but the same measurement on an untrained network locks harder. Claim retired
- A matched-energy spectral contrast is unachievable as designed, and a linear concept probe was retired after a fix for circularity left held-out R-squared negative, which rules out target leakage and leaves the concept absent from the learned state

One failure pattern recurred four times: a search that returns its own bound when it finds nothing, feeding an aggregate that cannot tell a bound from a measurement. I now check every derived scalar for what it returns when the quantity it measures is absent.

Software Engineering and Reproducibility

- A test-first codebase, currently 816 regression tests, with manifest-first reporting: every number in every report is read from a returned artifact, and a missing value renders as "N/A (not run)" rather than an estimate
- A frozen, versioned model zoo with SHA-256 integrity checks and a one-command prediction CLI, so teammates can run the model without touching training code
- SLURM array workflows, including a 160-shard saliency job whose resubmission script treats artifacts on disk as the source of truth, not the job accounting records

Fall 2026 Plans

The delay evidence so far shows the network depends on a delay I supplied. The Fall work asks a harder question: can the model find the physics itself, and does any of it survive contact with a real machine. Five directions, in priority order.

1. A learnable delay. The delay is currently hardcoded as the tooth-pass period. I will replace it with a differentiable delay kernel whose lag is a trained parameter, initialize it away from the physical value, and test whether gradient descent converges to the tooth-pass period from the signals alone. This is the strongest science claim available to the project: every delay result so far establishes dependence on a delay the model was given, whereas a learned delay landing on the physical one is discovery, and with the ablation and the SINDy-DDE recovery it would make three independent methods converging on one constant. The negative outcome is informative too: a lag settling elsewhere while accuracy holds means the model is on a proxy and the current interpretation needs revising. I will also compare a continuous-time neural DDE against the discrete recurrence, since the physics is continuous and the discretization is an untested assumption.

2. Multiplicative KAN nodes. The regenerative force is a product of depth of cut, cutting stiffness and the difference between current and delayed position. Additive splines can only approximate a product, and my symbolic extraction returns sums of univariate functions where the physics predicts one. KAN 2.0 multiplication nodes represent that form exactly, and the hypothesis is falsifiable two ways at once: a multiplicative trunk should reach the same AUC with fewer active edges, and its symbolic output should look like the physical term. This is the one architectural change in the project with a physical reason behind it.

3. Process damping. Seven hundred files are excluded because the analytic stability limit goes negative at low spindle speed, where the standard derivation breaks down and process damping dominates. Re-deriving the limit with a damping term would relabel them correctly. This is the only expansion of labeled data that does not depend on the data owner, and it targets exactly the regime the model has never seen. It also puts a real physics-residual term into the loss, a stub since the start for want of the constants, making the model physics-informed in its training objective and not only its architecture.

4. Prognosis. The model classifies a window as chattering or stable; the question a machinist asks is how much warning there is. I will relabel windows by time until onset within each file and measure a lead-time against accuracy curve, converting detection into forecasting. Detection is a crowded literature, onset prediction with a quantified lead time is not, and the severity head plus the conformal intervals already built express the answer as a calibrated warning. HAVOK intermittent-forcing analysis is the comparator, since it was designed to predict bursts in this kind of system.

5. Simulation to experiment. Every result here comes from a simulation database, and whether the method transfers to data recorded on a real machine decides whether this is a method or a method on simulation. I will fine-tune on a public experimental chatter dataset with a domain-discrepancy loss, then re-run the delay interventions on the adapted model. The interesting measurement is not transferred accuracy but whether the delay dependence survives adaptation: a model keeping its delay reliance on real signals is using physics that is genuinely there, one that loses it was fitting the simulator.

Alongside these I will set deterministic-algorithm flags in the trainer and re-run the two comparisons currently below the repeatability floor, complete the clean replication evaluation on datasets 126-155, publish the model zoo on Great Lakes for the two teammates who will use it, and produce the technical report and a manuscript draft.

Relevance to MS in Data Science

The project applies core MDS competencies: deep learning on sequences, physics-informed model design, interpretability, uncertainty quantification through conformal prediction, and experimental design with grouped cross-validation, leakage control and ablation. It runs on HPC throughout, with SLURM array workflows and GPU profiling driving the engineering decisions.

Beyond those competencies, the project treats its own reliability as a measurable quantity. I establish a repeatability floor before interpreting any difference in AUC, and I design interventions capable of retiring a claim, which an attribution plot by construction cannot do. Four claims in this project were retired on that evidence, and each retraction is documented in the weekly reports alongside the test that produced it.

_____

Prof. Barzan Mozafari (Faculty Advisor)